JShotz User Guide

Version 3.13.12

JShotz is a browser extension for Chrome, Edge, and Firefox that records a browsing flow as a sequence of timestamped, watermarked screenshots and exports them as a PDF, with an optional table of API calls captured under each step. It's built for documenting test flows, support tickets, and step-by-step evidence of what happened in a browser session.

Before you begin

- Record ordinary web pages. Browsers protect internal pages such as `chrome://` and extension store pages, so JShotz cannot inject its click and scroll capture helpers there.
- JShotz observes page fetch/XHR calls only during an **API + Screenshot** recording. Tab viewport and Screen/window modes leave site networking untouched.
- Keep the browser tab focused when using the manual `Ctrl+Alt+Q` shortcut.
- Treat **API + Screenshot** recordings as sensitive evidence. Request URLs, payloads, and responses can contain credentials, personal data, or other information that should not be shared outside the intended audience.

---

1. Installing the extension

Chrome / Edge

- Unzip `JShotz-3.13.12-chrome-edge.zip`.
- Go to `chrome://extensions` (or `edge://extensions`).
- Turn on **Developer mode** (top-right toggle).
- Click **Load unpacked** and select the unzipped folder that contains `manifest.json`.
- If updating, remove or disable the old version first so only one JShotz copy is loaded.

Firefox

- Unzip `JShotz-3.13.12-firefox.zip`.
- Go to `about:debugging#/runtime/this-firefox`.
- Click **Load Temporary Add-on** and select the `manifest.json` inside the unzipped folder.
- Firefox 128+ is required.

Once loaded, pin the JShotz icon to your toolbar for easy access.

Updating JShotz

After reloading or updating the extension, refresh every tab that was already open before you record with it. Closing and reopening the target tab is the most reliable option. Chrome keeps the old page script alive until the tab reloads and may show **Extension context invalidated** in the extension's error list. Delete that old error card after refreshing the tab; it does not indicate a problem with a newly opened or refreshed recording page.

---

2. The popup

Click the JShotz icon to open the popup. It has three parts:

- **Status line** — shows *Idle*, *Recording · N screenshot(s)*, *Paused · N screenshot(s)*,

or an error message.

- **Settings** — capture source and behavior options (locked once recording starts, except the

capture source, which can be changed mid-recording).

- **Actions** — Start/Stop, Pause/Continue, Capture now, Capture in 5s, Export PDF so far, and

Resume capture from folder.

- **Screenshots list** — a live list of what's been captured so far in the current session and

checkboxes that choose the screenshots included in a PDF.

Open this popup from the browser toolbar after loading JShotz as an extension. Opening `popup.html` directly from its source folder cannot start a recording because browser extension APIs are unavailable there.

---

3. Capture source (modes)

Mode	What it captures	Notes
Tab viewport only	The visible area of the recorded tab	Default mode, works everywhere
API + Screenshot	Same as Tab, plus a table of intercepted `fetch` /XHR calls under each screenshot	Shows request URL, origin/referer, payload, and response; failed calls are highlighted
Screen / window (DevTools + taskbar)	A shared screen, window, or tab surface via the browser's own share picker	The only mode that can show DevTools, the taskbar, or other applications

You can switch modes without stopping the recording. Pick a different option from the dropdown at any time. Switching *to* Screen/window mode opens a small picker window — click **Share** in it and choose what to share. The tab being recorded is automatically brought to

the front first, since the picker needs a real click.

If Screen/window mode's share source ends (you click "Stop sharing", or close the shared window), the recording automatically falls back to Tab-viewport captures rather than failing.

API mode begins collecting newly observed `fetch` and XHR calls while it is selected. It does not add network details retroactively to screenshots that were captured in another mode.

---

4. Settings

- **Capture on every button click** — automatically screenshot after clicks on buttons, links, checkboxes, and similar controls.
- **Capture every 60% of a screen scrolled, and at the end** — automatically screenshot as you scroll, roughly every 60% of a screenful (so consecutive shots overlap), plus one at the very bottom of the page.
- **Stamp clock + time zone on each shot** — adds a small timestamp banner to each screenshot.
- **Whole-page shot for Ctrl+Alt+Q and "Capture now" only** — see Section 6.
- **Save individual PNG files** — save each screenshot as its own PNG in the session folder.
- **Save PDF on stop** — build a PDF from the session when you stop and choose to keep it.

During a recording, the behavior checkboxes are locked so the session stays consistent. The **Capture source** menu remains available, allowing you to switch modes without ending the session.

---

5. Starting, capturing, and stopping

- Open the tab you want to record, then click **Start recording** in the popup.
- Use the page normally. Screenshots are taken automatically per your settings, and you can also trigger one manually at any time (see below).
- When a page you're recording opens a **new tab** (e.g. a sign-in redirect), JShotz follows it automatically — that tab is brought to the front and becomes part of the recording. Switching back to the original tab (or to any other tab that flow has opened) resumes capturing from wherever you actually are.
- To temporarily suspend screenshots without ending the session, click **Pause recording**.

The button changes to **Continue recording**. While paused, JShotz keeps the screenshot list, PDF selections, capture numbering, session folder, and tracked tabs or child windows. Click **Continue recording** to add subsequent screenshots to that same session.

If Chrome restarts during a recording, return to the restored page and open the JShotz popup. JShotz reconnects to that live tab, restores automatic and manual captures, and keeps the same screenshots, numbering, and session folder. Screen/window sharing cannot survive a browser restart, so that recording continues with tab-viewport capture until sharing is started again.

- When you're done, click **Stop recording**. Use the **Screenshots** list to clear any frames

you do not want in the PDF. **Select all** starts checked and automatically clears when any individual screenshot is unchecked. This choice remains for the current recording if the popup closes and is reopened. You'll then be asked whether to keep the files:
- **Yes, keep** — prompts for a PDF file name, then saves all PNGs and the manifest to your Downloads folder. The PDF contains only the screenshots that remain checked, and the folder opens when the export finishes.
- **No, delete all** — asks you to confirm, then removes everything from that session.

Manual capture options

Action	How	Notes
Capture now	Click **Capture now** in the popup	Immediate, uses whichever mode is active
Manual hotkey	Press **Ctrl+Alt+Q** while the page has focus	Works anywhere the page can receive keystrokes
DevTools panel capture	Press **Alt+Shift+S** while a DevTools panel is open	Captures exactly what's on screen, including the DevTools panel
Capture in 5s	Click **Capture in 5s**, or press **Alt+Shift+D**	Waits 5 seconds (with an on-page countdown badge) before capturing — use this when you need time to click into DevTools first, since Chrome blocks other shortcuts while DevTools has focus

Manual captures are available only while recording is active and not paused.

Export PDF so far

Click **Export PDF so far** at any point during a recording to write a checkpoint PDF from the screenshots currently checked in the **Screenshots** list, without stopping. The file is named `..._checkpoint.pdf` and does **not** open your Downloads folder automatically — only the final "Stop recording" export does that. The recording keeps going afterward, and the final PDF (on Stop) uses the screenshots selected at that time.

Resume capture from folder

Use this action when a browser or extension crash leaves a prior set of screenshots but the active recording cannot be recovered automatically:

- Open the page where the flow should continue.
- In the idle popup, click **Resume capture from folder**.
- Choose the exact earlier screenshot folder in the native folder dialog and grant read/write

access.

JShotz immediately reloads the **Screenshots** list from that folder and captures the current page with the next sequence number. The earlier and new screenshots are selected for **Export PDF so far **and the final** Stop recording** PDF. New PNGs, PDFs, and `flow-manifest.json` are written directly into the selected folder, and JShotz does not open a browser tab for this action.

This workflow needs Chrome or Edge's native File System Access API. Firefox can record normally, but cannot resume into an arbitrary existing folder.

After a browser restart, Chrome or Edge can require the folder permission again. JShotz changes the action to **Reconnect capture folder**; choose the same folder and the existing recording, screenshots, and sequence number continue unchanged.

---

6. Full-page (whole-page) capture

When **"Whole-page shot"** is enabled, the following triggers capture the *entire* scrollable page instead of just the visible area — **without visibly scrolling your screen**:

- **Ctrl+Alt+Q**
- **Capture now**
- **Capture in 5s**

Everything else (clicks, scrolling, field edits, navigation) stays as ordinary viewport shots, so your view is never disturbed by the automatic captures.

How it works, and what to expect:

- For a page whose content naturally extends below the viewport, Chromium rasterizes the document

beyond its existing viewport. JShotz does not enlarge or reflow the page to render the shot.

- For an "app-shell" style page (a fixed header/sidebar with an inner scrolling panel), JShotz

scrolls just that inner panel, then returns the panel to its original position. The surrounding page layout stays in place.

- Long captures are split into sequential, bounded **part N of M** screenshots at one shared

scale. Adjacent parts preserve the full page without a giant image, and are exported as consecutive PDF pages. Compact captures crop unneeded blank canvas conservatively.

- A progress bar appears in the popup and on the page while the capture runs. It is hidden before

each screenshot and removed when capture completes.

- If DevTools is already open on the tab, whole-page capture is skipped for that shot (DevTools

and the extension can't share the same debugging connection) and a normal single-frame screenshot is taken instead.

- Chrome may show a **"started debugging this browser"** banner for an ordinary-document capture.

This is a hard Chrome platform notice with no way to hide it; it disappears immediately after the shot. App-shell stitch capture does not attach the debugger.

---

7. Keyboard shortcuts

Shortcut	Action
`Ctrl+Alt+Q`	Manual capture (page must have focus)
`Alt+Shift+S`	Capture the current DevTools panel
`Alt+Shift+D`	Capture in 5 seconds

All three can be reassigned at `chrome://extensions/shortcuts` (or the Firefox equivalent).

---

8. PDF selection and saved files

Every capture starts selected for PDF output. Clear the checkbox beside an unwanted screenshot, or use **Select all** to restore the full set. For large sessions, the popup initially shows the 50 newest screenshots; use **Show older screenshots** to reveal earlier ones. The current selection is preserved while the session is active, even when the popup closes.

The same selection controls both **Export PDF so far** and the final PDF created through **Stop recording**. Individual PNG files and the final PDF depend on the saving options enabled when the session started:

- With **Save individual PNG files** enabled, JShotz saves each captured screenshot separately.
- With **Save PDF on stop** enabled, choosing **Yes, keep** creates the selected PDF after you enter its file name.
- With that PDF option disabled, keeping a session retains its available captured files but does not create a final PDF automatically.

When resuming a selected folder, JShotz enables both PNG and final-PDF saving so the older and new screenshots stay together in that folder.

The generated PDF uses the page title as the heading for each captured step. Timestamp banners appear only when **Stamp clock + time zone on each shot** was enabled. API tables appear only on screenshots captured in **API + Screenshot** mode.

9. Troubleshooting

- After installing, reloading, or updating JShotz, refresh any target tab that was already open.

This removes old injected page scripts and avoids an **Extension context invalidated** error.

- If the popup says JShotz is unavailable, reload the extension in the browser's extension page,

then close and reopen the popup.

- If the Screen/window share picker is cancelled or the shared source closes, recording continues

with Tab viewport captures. Select Screen/window again when you are ready to share a surface.

- During a whole-page capture, wait for the progress bar to complete before capturing again.

Capture now is temporarily disabled while that work is in progress.

- If a page does not record clicks or scrolling, confirm that it is an ordinary website rather

than a browser-protected page, then refresh the tab and start a new recording.

- If JShotz says **Capture skipped: JShotz needs access to the current page**, Chrome has revoked

its temporary tab access, commonly after a cross-site redirect. Open the JShotz popup in the current tab, then continue recording. For automatic capture across websites, open JShotz's extension details and set **Site access** to **On all sites**. The skipped-capture detail stays in the final manifest's `debugLog`; it does not create a separate download.

8. Where your files go

Each recording creates a folder in your browser's default download location:

```
Downloads/
  flow-captures/
    session_<timestamp>/
      001_<timestamp>_<label>.png
      002_<timestamp>_<label>.png
      ...
      flow-manifest.json        (list of every capture plus diagnostic debugLog)
      session_<timestamp>.pdf   (if "Save PDF" was on)
      session_<timestamp>_checkpoint.pdf   (if you used "Export PDF so far")
```

Pausing does not create another folder. When you click **Continue recording**, new screenshots keep the next number and are written beside the screenshots already shown in the list.

When you use **Resume capture from folder**, the selected existing folder becomes the working folder. JShotz writes the new PNGs, checkpoint PDF, final PDF, and updated `flow-manifest.json`

there instead of creating a new Downloads session folder.

---

9. The debug log

JShotz keeps the newest 1,000 diagnostic lines in background extension storage while recording. This prevents a separate diagnostic-file download or save prompt from interrupting pause, continue, capture, mode-switch, checkpoint-export, or error handling. When you keep a session, the same entries are included as `debugLog` in `flow-manifest.json` beside the screenshots. The log lists capture triggers, mode, success/failure, timing, mode switches, recovery events, and errors. If you ever need help diagnosing an issue, share the relevant `debugLog` lines from the final manifest.

If you choose "delete all," no diagnostic file is downloaded. The in-progress log remains in extension storage until you start a new recording.

---

10. Known limitations

- The "started debugging this browser" banner during whole-page captures cannot be hidden — this is a Chrome security notice, not a bug.
- Whole-page capture is skipped while DevTools is open on the recorded tab.
- Multi-tab and child-window following remains active while a session is paused and when the background service worker restarts.
- A horizontally-scrolling element on a page (e.g. a carousel) is captured exactly as it appears at the time — whole-page capture only extends vertically.
- Reloading or updating JShotz requires refreshing any already-open recording tabs before use.
- Resume capture from folder requires Chrome or Edge; Firefox does not provide writable native folder handles to extensions.

---

11. Getting help

If something isn't working as expected:

- Note the approximate time and what you were doing (which button, which page).
- Open `flow-manifest.json` from that session's folder and find the matching `debugLog` lines.
- Share the page URL (or a general description if it's private), the log excerpt, and — if relevant — the screenshot in question.